选这个：官方 ReAct Agent 模板

适合：零基础使用 Codex 跟做，先看懂一个小型 Agent 的完整执行过程。

推荐仓库：langchain-ai/react-agent
https://github.com/langchain-ai/react-agent

这是 LangChain 官方组织提供的 LangGraph ReAct Agent 模板。核心 graph.py 约 100 行，包含模型节点、工具节点、条件分支和循环。[1][2]

核心流程：用户提问 → 模型选择行动 → 执行工具 → 工具结果交回模型 → 继续调用或给出答案。

与截图中的 9 个名词怎么对应

概念	仓库里已有的能力 / 需要补的部分
Agent	已有：模型根据输入及工具结果决定下一步行动。
Tool Calling	已有：模型绑定工具，并生成结构化工具请求。
Function Calling	已有对应机制：工具请求包含函数名和参数，执行环境负责运行。
Executor	已有：ToolNode 执行工具，再将结果交回模型节点。
LangGraph	已有：通过 StateGraph、节点、条件边和回边组织流程。
Planner	模型承担行动决策；没有单独的 Planner 节点。不能把它等同于显式多步规划器。
Memory	README 描述了 Studio 同线程连续对话。长期偏好记忆需另行添加。
RAG	默认工具是 Tavily 联网搜索；本地文档检索不是现成能力，需要改造。
MCP	已检查的核心代码未接入；作为后续工具连接方式的扩展。

依据：项目 README、graph.py 和 tools.py；可点击的来源见第 3 页。此模板能帮助理解多个概念，但不代表开箱即用覆盖截图中的全部功能。

一小时，改成术语学习助手

目标：保留清晰的工具循环，加入本地资料检索，用真实日志理解概念。

第一小时只做这四步

时间	动作	可见产物
00-15 分钟	下载仓库、配置模型，检查当前运行方式。优先使用最简单的本地入口。	至少一次真实的模型工具调用。
15-35 分钟	把 Tavily 工具替换为 search_notes：读取本地 Markdown，关键词检索并返回片段和来源。	本地术语问答，答案可引用实际来源。
35-50 分钟	打印工具名、参数、真实结果；修改一条知识卡，再重复提问。	看清模型选择、工具执行与结果回传。
50-60 分钟	复盘图中术语，指出哪些已实现、哪些仍需扩展。	能够解释 Agent、RAG、工具调用与图编排。

把默认搜索换成小型 RAG

建一个 notes/ 文件夹，放 3 张知识卡：
rag.md：检索增强生成的定义与例子。
tools.md：工具调用、函数调用、规划与执行的区别。
memory.md：记忆、LangGraph、MCP 的位置。

search_notes(query) 按段检索，返回最多 3 条正分命中，附文件名、段落编号与原文。把片段交给模型生成回答；没有命中就说明找不到依据。

三项验收，别只看回答是否流畅

1. 工具确实执行：问“RAG 是什么”，日志里出现检索参数、真实结果和随后生成的回答。
2. 回答依赖资料：在笔记中添加虚构实验暗号，询问并修改暗号后再问，答案应跟随文件变化。
3. 无依据时停止编造：问笔记没有的信息，检索返回空结果，回答明确说明没有证据。

时间前提与下一轮

60 分钟是教学安排估计，默认已有可用 Python、网络及支持工具调用的模型 API；调用可能收费。此前只检查了仓库说明与代码，尚未在你的电脑上安装或运行验证。

长期偏好记忆、独立 Planner、MCP 接入放到下一轮。默认 Tavily 需要额外 API key；替换成本地检索后，可移除这项运行依赖。[1][3]

直接交给 Codex 的任务书

复制下面这段提示词，让编程助手围绕已选仓库推进。

请基于 https://github.com/langchain-ai/react-agent 帮我完成一个一小时入门项目。

先检查当前仓库代码、依赖和运行方式，在独立目录中操作，保留清晰的 LangGraph 模型与工具循环。模型使用我已有的 API 配置，先验证它支持真实 tool calling；不要硬编码或打印密钥。

把默认 Tavily 搜索替换为本地 Markdown 关键词检索，做成“Agent 术语学习助手”。检索返回命中片段、文件名和段落编号；回答只引用真实返回的来源。没有命中时说明缺乏证据。

提供最简单的命令行入口，打印节点、工具名、参数和实际结果；设置调用上限，避免无限循环。第一轮不增加网页界面、向量数据库、部署或多 Agent。

每完成一步，用中文解释它对应 Agent、Tool Calling、Function Calling、Executor、LangGraph 中的哪个概念。用虚构暗号的知识卡验证“资料改变，答案跟着改变”。

先跑通核心流程，给我运行命令与真实验收结果，再解释 Memory、Planner、MCP 应该在哪一层扩展。明确区分“已经实现”和“仅说明原理”，不要把计划当作成功结果。

复盘时，回答这三个问题

模型发出工具请求后，究竟是谁执行函数?
tools 节点之后为什么要回到模型节点?
换成 MCP Server 后，工具的连接方式与 Agent 的决策逻辑分别有什么变化?

来源与代码入口

[1] 仓库 README：定位、配置、Studio 交互与扩展方式
https://github.com/langchain-ai/react-agent#readme

[2] graph.py：模型调用、ToolNode、条件分支与回边
https://github.com/langchain-ai/react-agent/blob/main/src/react_agent/graph.py

[3] tools.py：默认 Tavily 搜索工具
https://github.com/langchain-ai/react-agent/blob/main/src/react_agent/tools.py

资料依据来自本次对话中对仓库 README 与核心代码的检查。GitHub 内容可能更新，实际安装时以当前仓库为准。时间安排与改造步骤是本方案建议。